# TCSS 487 — Cryptography

Practical Project — Cryptographic Library & App

Part 2: Elliptic Curve Arithmetic, ECIES, and Schnorr Signatures


| | |
|---|---|
| **Authors:** | Rudolf Arakelyan (rudik30), Linda Miao |
| **Course:** | TCSS 487 — Cryptography |
| **Project:** | Part 2 (depends on and includes Part 1) |
| **Curve:** | NUMS ed-256-mers* (Edwards form) |
| **Security level:** | 128-bit |
| **Language:** | Java (no external libraries beyond java.* ) |


## 1. Overview

This report documents Part 2 of the TCSS 487 cryptographic library and application. Part 2 builds the asymmetric layer on top of Part 1's SHA-3 / SHAKE implementation. We implement the NUMS-256 Edwards elliptic curve and use it to provide ECIES public-key encryption and elliptic Schnorr digital signatures at the 128-bit security level.

The application is a single-class command-line program. The user invokes **java Main <command> <args...>** to run any of the seven services described in Section 4.

### Files submitted

- **Main.java** — command-line dispatcher and all seven services.
- **Edwards.java** — NUMS-256 curve and the nested Point class (Appendix D spec).
- **SHA3SHAKE.java** — carried forward from Part 1 (FIPS 202, with byte-oriented sponge).
- **TCSS487_Project2_Report.pdf** — this report.

## 2. Cryptographic Design

### 2.1 Curve parameters

The curve is NUMS ed-256-mers*, an Edwards curve defined over the prime field $\mathbf{F_p}$ where:

- **p** = $2^{256} - 189$ (256-bit prime, satisfies $p \equiv 3 \bmod 4$)
- Curve equation: $\mathbf{x^2 + y^2 = 1 + d \cdot x^2 y^2}$ with **d = 15343**
- Group order: **n = 4r** where **r = $2^{254}$ − 8717531046210607367859464238084058606 7** (254-bit prime)
- Neutral element: **O = (0, 1)**; opposite of (x, y) is (–x, y)
- Generator **G = ($x_0$, $y_0$)** with $y_0 \equiv -4 \pmod p$ and $x_0$ the unique even square root

**Important distinction:** *coordinates* live mod **p**; *scalars* (private keys, nonces, signature components) live mod **r**. Mixing the two would break every operation, so the Edwards class exposes them as two separate constants and the rest of the code is careful to reduce by the correct modulus.

## 2.2 Point arithmetic

Edwards point addition is computed by the unified formula from the spec:

```
(x1,y1) + (x2,y2) =
  ( (x1*y2 + y1*x2) / (1 + d*x1*x2*y1*y2) ,
    (y1*y2 - x1*x2) / (1 - d*x1*x2*y1*y2) )   (all mod p)
```

All "divisions" are modular: the denominator is inverted with BigInteger.modInverse(p) and then multiplied. The formula is unified, meaning the same code handles P+Q, P+P, and P+(–P) with no special-case branching.

Scalar multiplication uses the left-to-right double-and-add algorithm from Appendix B. We scan the bits of the scalar m from most-significant to least-significant, doubling on every step and adding P when the current bit is 1. This runs in O(log m) point additions instead of O(m) — for a 254-bit scalar that is the difference between a few hundred additions and a number larger than the number of atoms in the observable universe.

## 2.3 Square root mod p (point decompression)

Public keys and ciphertext ephemeral points are transmitted in compressed form: only the y-coordinate and the LSB of the x-coordinate. Recovering x from y requires computing $\sqrt{((1 - y^2) / (1 - d \cdot y^2))}$ mod p. Because $p \equiv 3 \pmod 4$, we use the closed-form root from Appendix A:

```
r = v^((p+1)/4) mod p     // candidate root
if r.lsb != desired_lsb: r = p - r
if r*r mod p == v: return r
else: return null  // v is a quadratic non-residue
```

## 2.4 Key generation

From a passphrase, we derive the canonical private scalar s as follows:

- Init SHAKE-128, absorb passphrase bytes (UTF-8), squeeze 48 bytes (384 bits).
- Convert to BigInteger and reduce mod r.
- Compute V = s·G; if V.x is odd, replace $s \leftarrow r - s$ and $V \leftarrow -V$.

This LSB-flip step is critical — it guarantees that the public key always has an even x-coordinate, so transmitting only the LSB (always 0) is redundant but the same passphrase always yields the same s on both sides of an encryption/signing exchange. We factor this whole procedure into a single helper, derivePrivateKey(), used by keygen, decrypt, sign, signencrypt, and decryptverify.

## 2.5 ECIES encryption

For a recipient public key V and plaintext m:

- Pick random $k \Leftarrow$ SecureRandom (Appendix E recipe: 2·rbytes seed, reduce mod r).
- Compute W = k·V and Z = k·G.
- SHAKE-256.absorb(W.y); squeeze 32 bytes $\rightarrow \mathbf{k_a}$, then 32 bytes $\rightarrow \mathbf{k_e}$.
- $c = m \oplus$ SHAKE-128($k_e$) keystream of length |m|.

- t = SHA-3-256($k_a$ ■ c). *NB: the MAC is over the ciphertext c, per the spec's typo correction.*
- Output cryptogram: (Z, c, t), with Z compressed as (Z.y, Z.x & 1).

### 2.6 ECIES decryption

For a passphrase and cryptogram (Z, c, t):

- Recompute s from the passphrase (same derivePrivateKey).
- Decompress Z, compute W = s·Z. By the DH property s·(k·G) = k·(s·G) = k·V.
- Re-derive $k_a$ and $k_e$ from W.y exactly as in encryption.
- Compute t' = SHA-3-256($k_a$ ■ c). If t' ≠ t: abort, declare authentication error.
- Only when tags match: m = c ⊕ SHAKE-128($k_e$).

We check the tag **before** producing any plaintext output. A tampered or wrong-key ciphertext is rejected with a clear error message and no file is written.

### 2.7 Schnorr signatures

Signing a message m under passphrase:

- s ← derivePrivateKey(pass); pick random k mod r.

For Schnorr nonce generation, we used the basic SecureRandom approach from Appendix E. The hedged variant was noted but not implemented as the spec describes it as optional.

- U = k·G.
- h = SHA-3-256(U.y ■ m) mod r.
- z = (k – h·s) mod r.
- Signature = (h, z).

Verifying a signature (h, z) on m under public key V:

- U' = z·G + h·V.
- h' = SHA-3-256(U'.y ■ m) mod r.
- Accept iff h' = h.

### 2.8 Combined sign-then-encrypt (bonus)

The two bonus services compose the primitives above. signencrypt signs the plaintext with the sender's passphrase to produce (h, z), serializes message+h+z into one byte blob with ObjectOutputStream, and ECIES-encrypts the blob under the recipient's public key. decryptverify reverses the process: ECIES-decrypts with the recipient's passphrase, deserializes the message and signature, and Schnorr-verifies under the sender's public key. Importantly, the plaintext is **only written to disk if the signature verifies**, so an unauthenticated message can never silently reach the user.

## 3. File and Class Structure

All Java sources are in the default (unnamed) package as required, and the entry-point class is named **Main** in **Main.java**. There are exactly three Java source files:

| File | Contents |
|---|---|

| | |
|---|---|
| Main.java | main() dispatcher + 7 service methods + 5 small helpers |
| Edwards.java | Edwards curve class + nested Point class (Appendix D) |
| SHA3SHAKE.java | SHA-3 / SHAKE sponge (carried over from Part 1) |

## 4. User Instructions

All input/output filenames and the passphrase are supplied on the command line via String[] args, as required by the spec. Compile once, then invoke any service.

### 4.1 Build

```
# from the directory containing the .java files:
javac Main.java Edwards.java SHA3SHAKE.java
# or simply:
javac *.java
```

This produces Main.class, Edwards.class (and Edwards$Point.class), and SHA3SHAKE.class.

### 4.2 Services (command summary)

| Command | Arguments | What it does |
|---|---|---|
| **keygen** | <passphrase> <pubkeyfile> | Derive a key pair from the passphrase; write the public key to a file. |
| **encrypt** | <pubkeyfile> <inputfile> <outputfile> | ECIES-encrypt the input file under the given public key. |
| **decrypt** | <passphrase> <inputfile> <outputfile> | ECIES-decrypt the input cryptogram with the passphrase-derived private key. |
| **sign** | <passphrase> <inputfile> <sigfile> | Schnorr-sign the input file with the passphrase-derived private key. |
| **verify** | <pubkeyfile> <inputfile> <sigfile> | Verify a Schnorr signature on the input file under the given public key. |
| **signencrypt** | <senderpass> <recipientpub> <inputfile> <outputfile> | BONUS: sign with sender's pass, then ECIES-encrypt under recipient's key. |
| **decryptverify** | <recipientpass> <senderpub> <inputfile> <outputfile> | BONUS: ECIES-decrypt with recipient's pass, then Schnorr-verify under sender's key. |

### 4.3 Worked example (end-to-end session)

The following session demonstrates every service. Alice's passphrase is aliceSecret, Bob's is bobSecret.

```
# 1. Each party generates a key pair.
java Main keygen aliceSecret alice_pub.key
java Main keygen bobSecret bob_pub.key

# 2. Alice encrypts a message for Bob using his public key.
echo "Hello Bob, this is Alice." > note.txt
java Main encrypt bob_pub.key note.txt note.enc

# 3. Bob decrypts using his own passphrase.
java Main decrypt bobSecret note.enc note.out
cat note.out          # -> Hello Bob, this is Alice.

# 4. Alice signs the same file.
java Main sign aliceSecret note.txt note.sig

# 5. Anyone with Alice's public key can verify the signature.
```

```
java Main verify alice_pub.key note.txt note.sig
# -> Signature is VALID.

# 6. BONUS: Alice signs AND encrypts in one shot for Bob.
java Main signencrypt aliceSecret bob_pub.key note.txt sealed.bin

# 7. BONUS: Bob decrypts AND verifies in one shot.
java Main decryptverify bobSecret alice_pub.key sealed.bin opened.txt
# -> Decryption successful.
# -> Signature is VALID.
```

# 5. Testing

## 5.1 Curve arithmetic (Appendix C)

We exercised all the Appendix-C identities; every one passes:

| Identity | Result |
|---|---|
| `0 · G == O` | **PASS** |
| `1 · G == G` | **PASS** |
| `G + (–G) == O` | **PASS** |
| `2 · G == G + G` | **PASS** |
| `4 · G == 2 · (2 · G)` | **PASS** |
| `4 · G != O` | **PASS** |
| `r · G == O` | **PASS** |
| `k·G == (k mod r)·G  (random k)` | **PASS** |
| `(k+1)·G == k·G + G  (random k)` | **PASS** |
| `(k+l)·G == k·G + l·G  (random k, l)` | **PASS** |
| `k·(l·G) == l·(k·G) == (k·l mod r)·G` | **PASS** |
| `associativity of point addition` | **PASS** |

## 5.2 End-to-end functional tests

- keygen + encrypt + decrypt round-trips the original plaintext byte-for-byte.
- Wrong passphrase on decrypt is detected by MAC failure (“authentication tag mismatch”).
- Tampering with the ciphertext or message after signing causes verify to print INVALID.
- signencrypt + decryptverify round-trip succeeds; flipping the sender pubkey causes INVALID.
- Tested with empty files, ASCII text, and small binary files; all round-trip correctly.

# 6. Known Bugs and Limitations

**No functional bugs are known at the time of submission.** The known limitations are scope/design decisions rather than defects:

- **Whole-file in memory.** readFile/writeFile load the entire input into a byte array. This is fine for the file sizes expected in this course but would not stream multi-gigabyte inputs.

• **Not constant-time.** Scalar multiplication uses the simple double-and-add from Appendix B, whose execution time depends on the Hamming weight of the scalar. A production deployment would prefer Montgomery's ladder. The spec explicitly allows the simple variant.

• **SecureRandom.generateSeed.** Per Appendix E we use generateSeed(2*rbytes), which on some systems can block briefly if the entropy pool is low. This is the recommended recipe from the spec.

• **Public-key file format.** We use Java's ObjectOutputStream to store (y, x_lsb). The files are therefore only readable by another Java program. This was the simplest interoperable choice within the course's Java-only constraint.

## 7. Implementation Notes and Citations

• Square-root recipe (modPow with exponent (p+1)/4) is taken verbatim from Appendix A of the project handout.

• Double-and-add scalar multiplication follows the pseudocode in Appendix B of the project handout.

• Nonce generation follows Appendix E (SecureRandom.generateSeed with twice the byte count of r, then reduce mod r).

• MAC-over-ciphertext correction follows the explicit erratum in the project handout (DHIES paper's Figure 1 typo).

• SHA-3 / SHAKE implementation is the one we wrote for Part 1; it was structurally inspired by Markku-Juhani Saarinen's tiny_sha3 reference C code (cited inline in SHA3SHAKE.java). No Java SHA-3 code was copied.

• Curve parameters and algorithm structure derive from the NUMS curves papers cited in the handout (Bos et al., eprint 2014/130 and JCEN 2015).

• No JUnit was used. No package clauses are present. No .class, .jar, or .exe files are included in the submission ZIP.

*Submitted by Rudolf Arakelyan and Linda Miao for TCSS 487, Spring 2026.*